Review: Array, ArrayList, 2D Array, Traversal array/2D array and algorithms

1.

The following incomplete statement is intended to create an `ArrayList` named `theaterClub` to store `Student` objects. Assume that the `Student` class has been properly defined.

```
ArrayList<Student> theaterClub = /* missing code */;
```

Which of the following can replace `/* missing code */` so that the statement will compile without error?

(A) `new Student ArrayList()`

(B) `new ArrayList(Student)`

(C) `new ArrayList<Student>()`

(D) `new ArrayList<theaterClub>()`

2.

Consider the following code segment.

```
ArrayList<String> words = new ArrayList<String>();
words.add("jump");
words.add("right");
words.add("plan");
/* missing statement */
```

Which of the following can replace `/* missing statement */` so that `words` contains `["jump", "right", "laugh", "plan"]` after the code segment is executed?

(A)
```
words.add(2, "laugh");
```
(A)

(B)
```
words.add(3, "laugh");
```
(B)

(C)
```
words.set(2, "laugh");
```
(C)

(D)
```
words.set(3, "laugh");
```
(D)

3.

Consider the following code segment.

```
ArrayList<Double> dataValues = new ArrayList<Double>();
dataValues.add(9.84);
dataValues.add(8.93);
dataValues.add(7.65);
dataValues.add(6.24);
dataValues.remove(2);
dataValues.set(2, 7.63);
```

Which of the following represents the contents of `dataValues` after executing the code segment?

(A) `[9.84, 7.63, 6.24]`

(B) `[9.84, 7.63, 7.65, 6.24]`

(C) `[9.84, 8.93, 7.63]`

(D) `[9.84, 8.93, 7.63, 6.24]`

4.

The following method is intended to remove all elements from `words` whose length is greater than `max`.

```
/** Precondition: All elements of words are non-null. */
public static void removeLongWords(ArrayList<String> words,
                                   int max)
{
   /* missing code */
}
```

Which of the following can replace `/* missing code */` so that the method works as intended?

(A)

```
for (String w : words)
{
   if (w.length() > max)
   {
      words.remove(w);
   }
}
```

(B)

```
int i = 0;
for (String w : words)
{
   if (w.length() > max)
   {
      words.remove(w);
      i--;
   }
   i++;
}
```

(C)

```
for (int i = 0; i < words.size(); i++)
{
   if (words.get(i).length() > max)
   {
      words.remove(i);
   }
}
```

(D)

```
for (int i = words.size() - 1; i >= 0; i--)
{
   if (words.get(i).length() > max)
   {
      words.remove(i);
   }
}
```

5.

In the following code segment, assume that the `ArrayList` object `numbers` has been properly declared and initialized to contain `[0, 2, 4, 5]`.

```
for (int k = numbers.size() - 1; k >= 0; k--)
{
   if (numbers.get(k) > k)
   {
      System.out.print(k + " ");
   }
}
```

What, if anything, is printed as a result of executing the code segment?

(A) `2 4 5`

(B) `3 2 1`

(C) `5 4 2`

(D) Nothing is printed because an `IndexOutOfBoundsException` is thrown.

6.

In the following code segment, assume that the `ArrayList` `data` has been initialized to contain the `Integer` values `[4, 3, 4, 5, 3, 4]`.

```
int j = 0;
while (j < data.size() - 1)
{
   if (data.get(j) > data.get(j + 1))
   {
      System.out.print(data.get(j + 1) + " ");
   }
   j++;
}
```

What, if anything, is printed as a result of executing the code segment?

(A) `3 3`

(B) `4 5`

(C) `4 5 4`

(D) Nothing is printed because an `IndexOutOfBoundsException` occurs.

7.

Consider the following method `countNegatives`, which returns the number of elements in `numList` that are less than zero.

```
public static int countNegatives(ArrayList<Integer> numList)
{
   int count = 0;
   for (int j = 0; j < numList.size(); j++)       // Line 4
   {
      if (numList.get(j) < 0)
      {
         count++;
      }
   }
   return count;
}
```

Suppose the Boolean expression `j < numList.size()` in line 4 of the method is replaced with `j <= numList.size() - 1`. Which of the following best explains the effect of this change?

(A) It has no effect on the behavior of the method.

(B) It causes the method to ignore the last element in `numList`.

(C) It causes an `IndexOutOfBoundsException` to be thrown.

(D) It changes the number of times the loop executes, but all elements in `numList` will still be accessed.

8.

Consider the following code segment.

```
int count = 0;
for (String word : wordList)
{
   if (word.indexOf("a") >= 0)
   {
      count++;
   }
}
System.out.println(count);
```

Assume that the `ArrayList` `wordList` has been initialized to contain the `String` values `["apple", "banana", "coconut", "lemon", "orange", "pear"]`.

What is printed as a result of executing the code segment?

(A) 2

(B) 3

(C) 4

(D) 5

9.

Consider the following method, which is intended to remove duplicate consecutive elements contained in `nums`.

```
public static void remDups(ArrayList<Integer> nums)
{
   for (int j = 0; j < nums.size() - 1; j++) // Line 3
   {
      int first = nums.get(j);
      int second = nums.get(j + 1);
      if (first == second)                   // Line 7
      {
         nums.remove(j);                     // Line 9
         j++;                                // Line 10
      }
   }
}
```

For example, if `nums` contains `{1, 2, 2, 3, 4, 3, 5, 5, 6}`, then after `remDups(nums)` is executed, `nums` should contain `{1, 2, 3, 4, 3, 5, 6}`.

The method does not always work as intended.

Which of the following changes can be made so that the method always works as intended?

(A) In line 3, changing the Boolean expression to `j < nums.size()`

(B) In line 7, changing the Boolean expression to `first != second`

(C) In line 9, changing the expression to `nums.remove(j + 1)`

(D) In line 10, changing the expression to `j--`

10.

Assume that `arr` is a properly declared and initialized two-dimensional array of `String` objects with 3 rows and 4 columns. Which of the following statements can be used to print the four corner elements of `arr`?

(A)
```
System.out.print(arr[0, 0] + arr[0, 3] + arr[2, 0] + arr[2, 3]);
```
(A)

(B)
```
System.out.print(arr[1, 1] + arr[1, 4] + arr[3, 1] + arr[3, 4]);
```
(B)

(C)
```
System.out.print(arr[0][0] + arr[0][2] + arr[3][0] + arr[3][2]);
```
(C)

(D)
```
System.out.print(arr[0][0] + arr[0][3] + arr[2][0] + arr[2][3]);
```
(D)

11.

Consider the following method.

```
public static boolean checkIndices(double[][] data,
                                   int row, int col)
{
   int numRows = data.length;

   if (row < numRows)
   {
      int numCols = data[0].length;
      return col < numCols;
   }
   else
   {
      return false;
   }
}
```

Consider the following statement, which appears in a method in the same class as `checkIndices`.

```
double[][] table = new double[5][6];
```

Which of the following method calls will return the value `true`?

(A) `checkIndices(table, 4, 5)`

(B) `checkIndices(table, 4, 6)`

(C) `checkIndices(table, 5, 4)`

(D) `checkIndices(table, 5, 6)`

12.

Consider the following code segment, which is intended to declare and initialize the two-dimensional `String` array `things`.

```
/* missing code */ = {{"spices", "garlic", "onion", "pepper"},
                      {"clothing", "hat", "scarf", "gloves"},
                      {"plants", "tree", "bush", "flower"},
                      {"vehicles", "car", "boat", "airplane"}};
```

Which of the following could replace `/* missing code */` so that `things` is properly declared?

(A) `new String[][] things`

(B) `new(String[][]) things`

(C) `String[] String[] things`

(D) `String[][] things`

13.

Consider the following code segment.

```
String[][] words = {{"Hello,", "Hi,", "Hey,"},
                    {"it's", "it is", "it really is"},
                    {"nice", "great", "a pleasure"},
                    {"to", "to get to", "to finally"},
                    {"meet", "see", "catch up with"},
                    {"you", "you again", "you all"}};

for (int j = 0; j < words.length; j++)
{
   for (int k = 0; k < words[j].length; k++)
   {
      if (k == 1)
      {
         System.out.print(words[j][k] + " ");
      }
   }
}
```

What, if anything, is printed as a result of executing the code segment?

(A) Nothing is printed due to an `ArrayIndexOutOfBoundsException`.

(B)
```
Hello, it's nice to meet you
```

(C)
```
Hey, it really is a pleasure to finally catch up with you all
```

(D)
```
Hi, it is great to get to see you again
```

14.

Consider the following code segment.

```
int[][] arr = {{1, 2, 3},
               {4, 5, 6},
               {7, 8, 9},
               {3, 2, 1}};

for (int j = 0; j < arr.length; j++)
{
   for (int k = j; k < arr[0].length; k++)
   {
      System.out.print(arr[j][k] + " ");
   }
   System.out.println();
}
```

What is printed as a result of executing the code segment?

A

```
1 2 3
4 5
7
```

B

```
1 2 3
5 6
9
```

C

```
1 4 7
5 8
9
```

D

```
1 2 3
5 6
9
1
```

15.

Consider the following code segment, where `num` is an integer variable.

```
int[][] arr = {{11, 13, 14, 15},
               {12, 18, 17, 26},
               {13, 21, 26, 29},
               {14, 17, 22, 28}};

for (int j = 0; j < arr.length; j++)
{
   for (int k = 0; k < arr[0].length; k++)
   {
      if (arr[j][k] == num)
      {
         System.out.print(j + k + arr[j][k] + " ");
      }
   }
}
```

What is printed when `num` has the value `14`?

(A) `14 14`

(B) `16 17`

(C) `17 16`

(D) `18 19`

16.

Consider the following code segment.

```
int[][] data = {{5, 8, 6},
                {8, 2, 5},
                {9, 8, 8},
                {6, 8, 3}};

int result = data[0][0];
for (int col = 0; col < data[0].length; col++)
{
   for (int row = 0; row < data.length; row++)
   {
      if (result > data[row][col])
      {
         result = data[row][col];
      }
   }
}
```

What is the value of `result` after executing the code segment?

(A) 2

(B) 3

(C) 5

(D) 9

17.

In the following method, the parameter `temps` is a two-dimensional array of temperatures. The method is intended to return the number of temperatures in a rectangular portion of the array that are greater than 100. The portion is defined by rows `r1` to `r2`, inclusive, and columns `c1` to `c2`, inclusive.

```
/**
 *  Preconditions: 0 <= r1 <= r2 < temps.length
 *                 0 <= c1 <= c2 < temps[0].length
 */
public static int countHighTempsInRegion(int[][] temps,
                         int r1, int r2, int c1, int c2)
{
   int count = 0;
   /* missing code */
   return count;
}
```

Which of the following can replace `/* missing code */` so that the method works as intended?

(A)

```
for (int r = 0; r < temps.length; r++)
{
   for (int c = 0; c < temps[0].length; c++)
   {
      if (temps[r][c] > 100)
      {
         count++;
      }
   }
}
```

(B)

```
for (int r = r1; r < r2; r++)
{
   for (int c = c1; c < c2; c++)
   {
      if (temps[r][c] > 100)
      {
         count++;
      }
   }
}
```

(C)

```
for (int r = r1; r <= r2; r++)
{
   for (int c = c1; c <= c2; c++)
   {
      if (temps[r][c] > 100)
      {
         count++;
      }
   }
}
```

(D)

```
for (int r = r1; r <= r2; r++)
{
   for (int c = c1; c <= c2; c++)
   {
      if (temps[c][r] > 100)
      {
         count++;
      }
   }
}
```

18.

In the following method, the parameter `temps` is a two-dimensional array of temperatures. The method is intended to return the number of temperatures in a rectangular portion of the array that are greater than 100. The portion is defined by rows `r1` to `r2`, inclusive, and columns `c1` to `c2`, inclusive.

```
/**
 *  Preconditions: 0 <= r1 <= r2 < temps.length
 *                 0 <= c1 <= c2 < temps[0].length
 */
public static int countHighTempsInRegion(int[][] temps,
                         int r1, int r2, int c1, int c2)
{
   int count = 0;
   /* missing code */
   return count;
}
```

Which of the following can replace `/* missing code */` so that the method works as intended?

(A)

```
for (int r = 0; r < temps.length; r++)
{
   for (int c = 0; c < temps[0].length; c++)
   {
      if (temps[r][c] > 100)
      {
         count++;
      }
   }
}
```

(A)

(B)

```
for (int r = r1; r < r2; r++)
{
   for (int c = c1; c < c2; c++)
   {
      if (temps[r][c] > 100)
      {
         count++;
      }
   }
}
```

(B)

(C)

```
for (int r = r1; r <= r2; r++)
{
   for (int c = c1; c <= c2; c++)
   {
      if (temps[r][c] > 100)
      {
         count++;
      }
   }
}
```

(C)

(D)

```
for (int r = r1; r <= r2; r++)
{
   for (int c = c1; c <= c2; c++)
   {
      if (temps[c][r] > 100)
      {
         count++;
      }
   }
}
```

(D)

19.

Consider the following method, `countAs`, which is intended to traverse all the elements in the two-dimensional (2D) `String` array `things` and return the total number of elements that contain at least one `"a"`. The method does not always work as intended.

```
public static int countAs(String[][] things)
{
   int count = 0;
   for (int r = 0; r < things.length; r++)
   {
      for (int c = 0; c < things[r].length - 1; c++)
      {
         if (things[r][c].indexOf("a") >= 0)
         {
            count++;
         }
      }
   }
   return count;
}
```

For example, consider the following initialization.

```
String[][] order = {{"salad", "soup"},
                    {"water", "coffee"}};
```

The method call `countAs(order)` should return `2`.

For which of the following two-dimensional arrays arguments does `countAs` **not** work as intended?

(A)

```
{{"lemon"}, {"lime"}}
```

(B)

```
{{"rabbit", "bird"},
 {"cat", "dog"},
 {"gecko", "turtle"}}
```

(C)

```
{{"scarf", "gloves", "hat"},
 {"shoes", "shirt", "socks"}}
```

(D)

```
{{"math", "english", "physics"},
 {"golf", "baseball", "soccer"}}
```